UNIT 1

---

1. Define algorithm. List out the different design techniques. *(2M)*

An **algorithm** is a finite sequence of well-defined steps used to solve a problem or perform a task. It should be correct, efficient, and terminate after a finite number of steps.
Design techniques include: Divide and Conquer, Greedy Method, Dynamic Programming, Backtracking Algorithms, NP - Complete and NP - Hard.

---

2. Define worst case, average case and best case analysis. *(2M)*

- **Worst Case**: The maximum time an algorithm takes for any input.
- **Average Case**: The expected time taken, averaged over all possible inputs.
- **Best Case**: The minimum time taken by the algorithm for the most favorable input.

---

3. Define growth rate of function. *(2M)*

The **growth rate** of a function describes how the **resource requirements** (like time or space) of an algorithm increase with the input size *n*.
It is used to compare the efficiency of algorithms by focusing on scalability and performance for large inputs.

---

4. Differentiate between algorithm and program. *(2M)*

An **algorithm** is a step-by-step logical procedure to solve a problem, independent of any programming language.
A **program** is an implementation of an algorithm in a specific programming language.
While algorithms are theoretical, programs are executable.

---

5. Differentiate between recursive and iterative algorithm design. *(2M)*

A **recursive algorithm** solves a problem by calling itself with smaller inputs until a base case is reached.
An **iterative algorithm** uses loops and does not involve self-calls.
Recursion is often simpler to write, but iteration is generally more space-efficient.

---

6. List and explain different types of complexities used to find efficiency of algorithm. *(2M)*

- **Time Complexity**: Time taken by the algorithm to run as a function of input size.

- **Space Complexity**: Amount of memory used during execution.
  These help in analyzing the efficiency of algorithms and making optimal choices.

---

7. Define Asymptotic Notation. *(2M)*

Asymptotic notations describe the limiting behavior of an algorithm's time or space complexity as input size approaches infinity.
It helps ignore constant and lower-order terms.
Common notations:

- **Big-O (O)** – Upper bound (worst-case)
- **Omega (Ω)** – Lower bound (best-case)
- **Theta (Θ)** – Tight bound (average-case)

---

8. Explain the characteristics of an algorithm. *(2M)*

1. **Finiteness** – It must terminate after a finite number of steps.

2. **Definiteness** – Each step must be clear and unambiguous.

3. **Input & Output** – It should accept input(s) and produce output(s).

4. **Effectiveness** – Steps must be basic enough to be performed easily.

9. Differentiate Time Efficiency and Space Efficiency. *(2M)*

- **Time Efficiency** refers to how fast an algorithm runs.

- **Space Efficiency** measures the amount of memory it uses during execution.
  Efficient algorithms balance both to optimize performance.

---

10. Explain recursion with example. *(2M)*

Recursion is a method where a function calls itself to solve a smaller instance of the same problem. Types of recurrence relations:

- Homogeneous recurrence
- Non - homogeneous recurrence
- Logarithmic recurrence

Example:

```
int factorial(int n) {
   if(n == 0) return 1;
   return n * factorial(n - 1);
}
```

---

11. Define Amortized Analysis. List out the method of Amortized Analysis. *(2M)*

Amortized Analysis gives the **average time per operation** over a sequence, smoothing out expensive operations.
It's useful when some operations are costly but occur infrequently.
Methods include:

- Aggregate Method
- Accounting Method
- Potential Method.

---

12. Discuss the need of Analysing Algorithms. *(2M)*

Analyzing algorithms helps in evaluating their **efficiency** in terms of **time** and **space** before actual implementation.
It enables comparison between different algorithms to choose the most optimal one for a given problem.
Understanding algorithm analysis also ensures better performance and scalability for large input sizes. This is done by analysing time and space complexity.

---

13. Define Asymptotic Notations. Explain different types of Asymptotic Notations. *(4M)*

Asymptotic notations are used to describe the **growth rate of an algorithm's running time** or space requirement as input size increases.
They help in comparing algorithms by ignoring constant factors and lower-order terms.

Types of Asymptotic Notations:

1. **Big O (O)** – Describes the upper bound (worst-case).
   Example: $O(n^2)$ – time will not exceed this.

2. **Omega (Ω)** – Describes the lower bound (best-case).
   Example: $\Omega(n)$ – time will take at least this much.

3. **Theta (Θ)** – Describes the tight bound (average-case or exact).
   Example: $\Theta(n \log n)$ – time grows exactly like this.

---

14. Explain the characteristics of an algorithm. *(4M)*

The five key characteristics of a good algorithm are:

1. **Finiteness:** The algorithm must always terminate after a finite number of steps.

2. **Definiteness:** Every step must be clearly and unambiguously defined.

3. **Input:** The algorithm can have zero or more inputs.

4. **Output:** It must produce at least one output as the result.

5. **Effectiveness:** All operations must be basic enough to be performed exactly and in a finite amount of time.

---

15. “Constants are ignored in analysis of algorithm during complexity computation.” Justify with example. *(4M)*

In asymptotic analysis, we focus on the growth rate of an algorithm rather than exact execution time.
Constants are ignored because they have negligible impact on performance as input size grows large.

Example:
An algorithm with time complexity **T(n) = 3n + 10** simplifies to **O(n)** in asymptotic analysis.
Even if another algorithm has **T(n) = 5n + 100**, both are considered O(n) because their linear growth dominates.

Ignoring constants helps in comparing algorithms based on structure and scalability, not machine or language-specific details.

---

16. Define the term Recursion. *(2M)*

Recursion is a technique in which a function **calls itself** directly or indirectly to solve a problem by breaking it into smaller subproblems.
Every recursive function has a **base case** to terminate and a **recursive case** to call itself.

---

17. Define algorithm. How to measure an algorithm's running time? *(4M)*

An **algorithm** is a finite and ordered set of instructions to solve a particular problem or perform a computation.

Measuring running time of an algorithm involves analyzing:

1. **Input Size (n):** The size of the problem to be solved.

2. **Number of Basic Operations:** Counting the fundamental operations (comparisons, additions, etc.).

3. **Asymptotic Analysis:** Using notations like **Big O, Θ, Ω** to describe time complexity for best, worst, or average cases.
   This allows comparison of algorithm efficiency independent of programming language or hardware.

---

18. List and explain different types of complexities used to find efficiency of algorithm. *(4 M)*

There are two primary types of complexities used to evaluate an algorithm's efficiency:

1. **Time Complexity:**
   - Measures the **amount of time** an algorithm takes to complete, as a function of the input size (*n*).
   - It helps identify how fast an algorithm is and is commonly expressed using **asymptotic notations** like O(n), O(log n), etc.
2. **Space Complexity:**
   - Refers to the **amount of memory** an algorithm uses during its execution.
   - It includes memory for variables, input storage, function calls (especially in recursion), and auxiliary data structures.

Both complexities help in choosing the most suitable algorithm based on resource constraints and performance requirements.

---

19. Discuss the need of Analyzing Algorithms. *(4 M)*

Analyzing algorithms is essential to determine their **efficiency**, especially for large inputs. It helps in the following ways:

1. **Predict Performance:** Allows us to estimate the time and space usage without running the program.

2. **Compare Alternatives:** Helps in choosing the most suitable algorithm among multiple options for a given problem.

3. **Scalability Check:** Identifies how the algorithm performs as input size increases, which is crucial for large-scale applications.

4. **Resource Optimization:** Ensures minimal usage of computational resources, leading to better performance on limited hardware.

Thus, algorithm analysis is crucial for writing efficient, reliable, and scalable software.

---

20. Explain the Amortized Analysis and different methods used in Amortized Analysis. *(7 M)*

Amortized Analysis is a technique used to **analyze the average time per operation** over a sequence of operations, even if some individual operations are expensive.
Unlike worst-case analysis which considers the cost of the most expensive operation, amortized analysis provides a more realistic measure by distributing the total cost across all operations.

Need for Amortized Analysis:

In some algorithms, a single operation might be costly, but if it occurs rarely, the average cost is low.
A common example is **dynamic array resizing** — most insertions are O(1), but occasionally resizing takes O(n).

Methods of Amortized Analysis:

1. **Aggregate Method:**

    - Calculates the **total cost** of *n* operations and then divides by *n* to find the average cost.

    - Example: Inserting elements into a dynamic array with occasional resizing.

2. **Accounting Method:**

- Assigns **different charges** (amortized cost) to operations — more than actual cost for some, less for others.
- Surplus from cheaper operations is saved as “credit” for costly ones.

3. **Potential Method:**
   - Uses a **potential function** that maps the state of the data structure to a real number (potential).
   - The change in potential reflects stored work that can offset future expensive operations.

Example:

Consider a dynamic array that doubles in size when full. Although resizing is O(n), if we perform *n* insertions, the total cost is O(n), resulting in O(1) amortized time per insertion.

Thus, amortized analysis is especially useful for evaluating data structures like dynamic arrays, stacks with multipop, and splay trees.

UNIT 2

---

1. Describe the steps of Divide and Conquer Strategy. (2 M)

The Divide and Conquer strategy works by breaking a problem into **smaller subproblems**, solving each recursively, and then **combining** their results.
The three main steps are:

1. **Divide** the problem into smaller subproblems.
2. **Conquer** the subproblems by solving them recursively.
3. **Combine** the results to get the final answer.

---

2. Write the complexity of a) Selection sort b) Merge sort. (2 M)

a) **Selection Sort:** Time Complexity = **$O(n^2)$** in all cases.
b) **Merge Sort:** Time Complexity = **O(n log n)** for best, average, and worst cases.

---

3. Define External Path Length and Internal Path Length. (2 M)

- **External Path Length (EPL):** The sum of the path lengths from the root to all external (leaf) nodes in a binary tree.

- **Internal Path Length (IPL):** The sum of the path lengths from the root to all internal (non-leaf) nodes in a binary tree.
  These are important in evaluating tree-based structures like Huffman Trees.

---

4. Explain the steps of sorting array using Merge Sort. (2 M)

Merge Sort follows these steps:

1. **Divide** the array into two halves.

2. **Recursively sort** both halves.

3. **Merge** the sorted halves into a single sorted array.
   This process continues until the entire array is sorted.

---

6. Write an algorithm for Binary Search. (4 M)

Binary Search Algorithm (Iterative Version):

```
BinarySearch(arr, key)
1. Initialize low = 0, high = length(arr) - 1
2. while low ≤ high:
3.     mid = (low + high) / 2
4.     if arr[mid] == key:
5.         return mid
```

```
    else if arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
return -1  // Key not found
```

Explanation: Binary Search works on a **sorted array** and repeatedly divides the search interval in half, offering a time complexity of **O(log n)**.

---

12. Explain the Divide and Conquer Strategy in short. (4 M)

The **Divide and Conquer** strategy solves complex problems by breaking them down into **smaller, more manageable subproblems**. These are solved recursively, and their solutions are **combined** to solve the original problem.

Key Examples:

- **Binary Search** – Divides the array to find an element.
- **Merge Sort** – Recursively divides and merges the array.
- **Strassen's Matrix Multiplication** – Multiplies matrices using fewer multiplications by dividing them.
- **Min-Max Algorithm** – Finds minimum and maximum values using fewer comparisons.

It improves algorithm efficiency and is commonly used in recursive problem-solving.

---

16. Differentiate between Divide & Conquer and Dynamic Programming (4 M)

Feature	Divide and Conquer	Dynamic Programming
Approach	Breaks the problem into **independent subproblems**, solves them separately, and combines the results.	Breaks the problem into **overlapping subproblems**, solves each subproblem once, and stores the result.
Overlapping Subproblems	**Does not** store results of subproblems; may recompute them.	**Stores** results to avoid recomputation (uses memoization or tabulation).
Optimal Substructure	Required — problem must be solved using solutions to subproblems.	Also required — final solution is built using solutions to subproblems.
Examples	Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication	Fibonacci Sequence, Matrix Chain Multiplication, 0/1 Knapsack, Longest Common Subsequence
Efficiency	Can be inefficient for overlapping subproblems due to repeated computations.	Efficient for overlapping subproblems; avoids recomputation via reuse of stored results.

UNIT 3

---

1. List the advantage of the greedy algorithm. (2 M)

- Greedy algorithms are **simple to implement** and often lead to faster solutions.
- They **require less memory** as they do not store intermediate results like dynamic programming.
- They work well for problems where a **greedy choice property and optimal substructure** are present (e.g., Huffman Coding, Prim's MST).

---

2. List out the methods for solving Knapsack Problem. (2 M)

The three primary methods used to solve the Knapsack Problem are:

- **Based on maximum profit:** Select items with the highest profit first, without considering weight.
- **Based on minimum weight:** Choose items with the lowest weight first.
- **Based on maximum profit / weight ratio:** Select items with the highest profit-to-weight ratio.

Among these, the **profit/weight ratio** method is the most efficient for solving the **Fractional Knapsack Problem**.

---

4. Write the difference between a breadth-first search and a depth-first search. (2 M)

BFS (Breadth-First Search)	DFS (Depth-First Search)
Explores all neighbors before going deeper.	Explores as deep as possible before backtracking.

Uses a **queue**.	Uses a **stack** (or recursion).
Suitable for **shortest path** in unweighted graphs.	Suitable for **path finding** and **cycle detection**.

---

5. Explain Spanning tree and Minimum Cost Spanning Tree with example. (2 M)

- A **Spanning Tree** of a graph connects all its vertices with the **minimum number of edges** and **no cycles**.
- A **Minimum Cost Spanning Tree (MST)** is a spanning tree where the **sum of edge weights is minimized**.
- **Example:** For a weighted graph with edges (A-B, 4), (B-C, 1), (C-A, 3), MST includes edges B–C and C–A with total weight = **4**.

---

6. Define Minimum Cost Spanning Tree with example. (2 M)

A **Minimum Cost Spanning Tree (MST)** connects all vertices of a graph with the **least possible total edge weight**, and **no cycles**.
Example: Given a graph with edges: (A–B:1), (B–C:2), (A–C:3), the MST would include edges A–B and B–C, with cost **3**.

---

7. List out the three methods of solving Knapsack Problem. (2 M)

The three primary methods used to solve the Knapsack Problem are:

- **Based on maximum profit:** Select items with the highest profit first, without considering weight.

- **Based on minimum weight:** Choose items with the lowest weight first.
- **Based on maximum profit / weight ratio:** Select items with the highest profit-to-weight ratio.

Among these, the **profit/weight ratio** method is the most efficient for solving the **Fractional Knapsack Problem**.

---

9. Discuss the characteristics of Greedy Strategy. (4 M)

Greedy strategy has the following important characteristics:

1. **Greedy Choice Property:** A globally optimal solution can be arrived at by making a **locally optimal choice** at each step.
2. **Optimal Substructure:** The problem can be broken down into **smaller subproblems**, and solutions to subproblems help build the final solution.
3. **No Backtracking:** Once a choice is made, it is never reconsidered.
4. **Faster Execution:** Often faster and simpler compared to dynamic programming when applicable (e.g., Huffman Coding, Prim's Algorithm).
   These characteristics make the greedy approach suitable for problems like MSTs, Huffman Coding, and Job Sequencing.

---

13. Differentiate between the following. (7 M)

i) Greedy vs Dynamic Programming:

Greedy	Dynamic Programming
Makes **locally optimal** choices at each step.	Solves **overlapping subproblems** and stores results.

Does **not** always guarantee optimal solution.	**Always guarantees** optimal solution (if applicable).
Faster and simpler.	Slower, but more reliable for complex problems.
Used in problems like **Fractional Knapsack**, **Prim's Algorithm**.	Used in problems like **0/1 Knapsack**, **Matrix Chain Multiplication**.

---

ii) Homogeneous vs Non-Homogeneous Recurrences:

Homogeneous Recurrence	Non-Homogeneous Recurrence
All terms are dependent on the recursive function.	Contains additional **non-recursive** terms.
Form: $T(n) = aT(n/b) + c$	Form: $T(n) = aT(n/b) + f(n)$
Example: $T(n) = 2T(n/2)$	Example: $T(n) = 2T(n/2) + n$

---

iii) Divide and Conquer vs Dynamic Programming:

Divide and Conquer	Dynamic Programming

Solves **independent** subproblems.	Solves **overlapping** subproblems.
No storage of results.	Stores solutions to subproblems (memoization/table).
Example: Merge Sort, Quick Sort.	Example: 0/1 Knapsack, Fibonacci.
May repeat calculations.	Avoids repeated calculations.

---

18. Define Greedy Algorithm. (2 M)

A **Greedy Algorithm** builds a solution step-by-step, always choosing the **best option available at that moment**, hoping this leads to a globally optimal solution.
It is efficient for problems that exhibit **greedy choice property** and **optimal substructure**, like Prim's and Kruskal's algorithms.

---

19. State the Principle of Job Sequencing with Deadline. (2 M)

In **Job Sequencing with Deadline**, the objective is to **schedule jobs** within their respective deadlines such that the **total profit is maximized**.
Only one job can be done at a time, and if a job is completed within its deadline, its **full profit is earned**. Otherwise the **profit becomes invalid** when the **deadline is crossed**

---

20. Define Version Space. (2 M)

Version Space is the set of all hypotheses **consistent with the observed training examples**. It narrows down the hypothesis space by eliminating any function that doesn't match the input-output pairs.

---

27. Explain the different types of Graph traversal techniques. (4 M)

Graph traversal is a way of **visiting all the nodes (vertices)** of a graph. Two main techniques are:

1. **Breadth-First Search (BFS):**
    - Explores neighbors first, then their neighbors.
    - Uses a **queue**.
    - Suitable for **shortest path** in unweighted graphs.
2. **Depth-First Search (DFS):**
    - Explores as far as possible along a branch before backtracking.
    - Uses a **stack** or recursion.
    - Useful for **cycle detection**, **topological sorting**, and pathfinding.

Both BFS and DFS are fundamental in solving graph-related problems like connectivity, shortest path, and component detection.

UNIT 4

---

1. Write the principle of Dynamic Programming. (2 M)

The principle of **Dynamic Programming (DP)** is to solve a problem by breaking it down into **smaller overlapping subproblems**, solving each subproblem **only once**, and storing its result for future reference.
This avoids redundant computation and significantly reduces time complexity, especially in **optimization problems**.

3. State the principle of Backward and Forward Multistage Graph. (2 M)

In **Multistage Graph problems**, the goal is to find the **minimum cost path** from the source to the destination.

- **Forward Approach**: Start from the source and move stage by stage towards the destination, computing the minimum cost at each stage i.e. generating path from source to destination by using the information about stage i - 1 while calculating for stage i.

- **Backward Approach**: Start from the destination and move backward to the source, calculating minimum cost paths in reverse, i.e. generating path between source and destination by using the information about stage i + 1 while calculating for stage i.
  Both use **dynamic programming** to store intermediate costs for efficient computation.

---

4. Write difference between top-down and bottom-up dynamic programming approaches. (2 M)

Top-Down Approach	Bottom-Up Approach
Uses **recursion + memoization**.	Uses **iteration + tabulation**.
Starts solving from the **main problem** and breaks into subproblems.	Solves **all subproblems** first and uses them to build the final solution.
Better for problems with few relevant subproblems.	More efficient in terms of memory and speed for many subproblems.

---

5. Differentiate between Dynamic Programming and Divide & Conquer Strategy. (4 M)

Dynamic Programming (DP)	Divide and Conquer (D&C)
Solves problems with **overlapping subproblems** and **optimal substructure**.	Solves problems by dividing them into **independent subproblems**.
Stores solutions to subproblems to avoid recomputation (memoization/tabulation).	**Does not store** results; recomputes them when needed.
More suitable for **optimization problems** like 0/1 Knapsack, LCS.	Used for problems like Merge Sort, Quick Sort.
Time complexity is generally **lower** due to reuse of subproblem results.	Can be **less efficient** if the same subproblem is solved multiple times.

UNIT 5

---

1. Write the difference between BFS and DFS. (2 M)

BFS (Breadth-First Search)	DFS (Depth-First Search)
Explores neighbors level-by-level using a **queue**.	Explores as deep as possible using a **stack/recursion**.
Better for **shortest path** in unweighted graphs.	Useful for **cycle detection** and **connected components**.

---

2. Explain the Backtracking with example. (2 M)

Backtracking is a recursive algorithmic technique that tries to build a solution incrementally and abandons (or backtracks) as soon as it determines a partial solution cannot lead to a valid full solution.
Example: In the **N-Queens Problem**, we place queens one row at a time and backtrack if a queen is attacked, trying the next safe column.

---

3. Define Graph Colouring with example. (2 M)

Graph Coloring is the process of assigning colors to the vertices of a graph so that no two adjacent vertices share the same color.
Example: In a triangle graph (3 connected vertices), you need **3 different colors** to ensure adjacent vertices are not the same.

---

4. Explain Promising and Non-Promising Node. (2 M)

In **backtracking**, a node is **promising** if it **can possibly lead** to a solution based on current partial decisions.
A node is **non-promising** if it **violates** the problem's constraints early, and hence we **backtrack** from it to save time.

Example: In N-Queens, placing a queen in a position where it is under attack would make it a non-promising node.

---

5. Define tractable and non-tractable problems? (2 M)

- **Tractable Problems** are those that can be solved in **polynomial time** (e.g., $O(n^2)$, $O(n^3)$), meaning they're practically solvable.

- **Non-Tractable Problems** (intractable) require **exponential time** (e.g., $O(2^n)$) and become **impractical** as input size grows.
  Example: Sorting is tractable, while solving TSP by brute force is non-tractable.

---

6. Discuss the classes P, NP, NP-Complete, and NP-Hard with examples. (4 M)

Understanding the complexity classes **P, NP, NP-Complete**, and **NP-Hard** is essential for analyzing whether a problem can be solved efficiently.

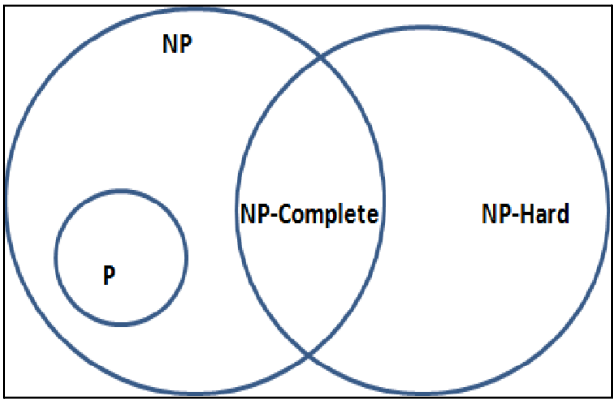
- **P (Polynomial Time):**
  This class includes all decision problems (yes/no type problems) that can be **solved in polynomial time** by a deterministic machine (i.e., an ordinary computer following standard steps). These are considered **efficient and tractable** problems.
  Example: Binary Search, Merge Sort, and Matrix Multiplication.

- **NP (Nondeterministic Polynomial Time):**
  NP includes problems for which, **if given a solution**, it can be **verified in polynomial time**, even if we don't know how to solve it efficiently ourselves. These problems may not necessarily be solvable quickly, but checking a given solution is fast.
  Example: Verifying a solution to a Sudoku puzzle or a solution to the Traveling Salesman Problem.

- **NP-Complete:**
  These are the **"hardest" problems in NP**. A problem is NP-Complete if it is **both in NP** and **as hard as any problem in NP**. If any one NP-Complete problem can be solved in polynomial time, then **all NP problems can be solved in polynomial time**, which would mean **P = NP**.
  Example: 0/1 Knapsack Problem, Boolean Satisfiability Problem (SAT), and Graph

Coloring.

- **NP-Hard:**
  These are problems that are at least as hard as NP-Complete problems, but they **do not have to be in NP**, meaning their solutions may not even be verifiable in polynomial time. They are **not necessarily decision problems**.
  Example: The Optimization version of the Traveling Salesman Problem and the Halting Problem.



---

8. State Cook's Theorem and explain it. (4 M)

Cook's Theorem is a fundamental result in computational complexity theory and forms the basis of the NP-Completeness concept.

- **Statement:**
  Cook's theorem states that **"satisfiability is in P if and only if P = NP"**.
  This means SAT is both in NP and that **every problem in NP can be reduced to SAT in polynomial time**.

- **Explanation:**
  Cook's Theorem was proved by Stephen Cook in 1971. It was the first problem shown to be NP-Complete, making it a **benchmark** for comparing the complexity of other NP problems.
  It implies that if we could find a **polynomial-time algorithm for SAT**, we could solve **all NP problems** in polynomial time — thus proving **P = NP**.
  The reduction from any NP problem to SAT is **constructive**, meaning we can transform an instance of any NP problem into an instance of SAT in polynomial time.

9. Write the algorithm for non-deterministic searching. (4 M)

A **non-deterministic algorithm** is one that can make “guesses” and verify solutions efficiently.

Algorithm:

```
Algorithm NDA_search(A, n)
{
    j := choice(1...n);
    if A[j] = x then
    {
        write(j);
        success();
    }
    write(0);
    failure();
}
```

Example: For SAT, guess a truth assignment and verify whether it satisfies the Boolean formula.

---

11. Justify the statement: “Satisfiability is in P if and only if P = NP”. (7 M)

Cook's theorem states that **"satisfiability is in P if and only if P = NP"**.

The **satisfiability problem** is to determine whether a formula is true for some assignment of truth values to the variables.

It is easy to obtain **polynomial-time, non-deterministic algorithms** that terminate successfully if a given propositional formula is satisfiable. To show **P = NP**, we obtain, from a polynomial-time, non-deterministic decision algorithm **A** and input **I**, a formula **φ(A, I)** such that **φ is satisfiable if and only if A successfully terminates with input I**.
If the length of **I** is **n**, and the time complexity of **A** is **T(n)** for some polynomial **p(n)**, then **φ** can be constructed.

A **deterministic algorithm 'z'** to determine the outcome of **A** on input **I** can be easily obtained. Algorithm **z** simply computes **φ** and then uses a **deterministic algorithm for satisfiability** to determine whether **φ** is satisfiable.

If **satisfiability is in P**, then every **non-deterministic algorithm A in NP** can obtain a **deterministic algorithm z** in **P**.

From the above statement, **Cook's theorem is proved**.
Hence, **P = NP if and only if satisfiability is in P**.

---

13. Write an algorithm for the following: (7 M)

i) Clique Problem

A **clique** is a subset of vertices in a graph such that every two distinct vertices are connected by an edge.

Algorithm:

```
Algorithm Clique(G, n, k, S)
{
    // Size of clique = k
    // S represents a set in which vertices are sorted.

    S = ∅  // S is initially empty set

    for i = 1 to k do
    {
        t = choice(1...n)

        if (t ∈ S) then
            failure();
        else
            S = S ∪ {t}  // t distinct vertices are selected
    }

    // At this point S contains k distinct vertex indices
    for all pairs (i, j) such that i ∈ S, j ∈ S and i ≠ j do
    {
        if ((i, j) ∉ E) then
            failure();
    }
```

```
    success();
}
```

---

ii) Graph Partitioned in Triangle

Find whether the graph can be divided into triangles (sets of 3 mutually connected vertices).

Algorithm:

```
Algorithm GPT(G, n, k, S)
{
    S ≠ ∅  // Initially set is empty

    for i = 1 to k do
    {
        t = choice(1...n)
        if (t ∈ S) then
            failure()
        else
            S = S ∪ {t}  // Add t to S
    }

    for each pair (i, j) ∈ S and i ≠ j do
    {
        if ((i, j) ∉ E) then
            failure();
    }

    success();
}
```

---

iii) Independent Set Problem

An **independent set** is a set of vertices in a graph, no two of which are adjacent.

Algorithm:

```
Algorithm ISP(G, n, t, S₁, S₂)
{
    // t → size of S₁
    S₁ = ∅, S₂ = ∅

    for i = 1 to t do
    {
        j = select(1...n)

        if (j ∈ S) then
            failure();
        else
        {
            S₁ = S₁ ∪ {j}
            S₂ = G - S₁  // Remaining vertices
        }
    }

    // Connectivity check
    for i = 1 to t do  (i ∈ S₁)
        for j = 1 to (G - t) do  (j ∈ S₂)
        {
            if ((i, j) ∉ E) then
                failure();
        }

    success();
}
```

16. Write an algorithm for non-deterministic searching and non-deterministic sorting. (8 M)

A. Non-Deterministic Searching Algorithm

A non-deterministic search algorithm "guesses" the solution and verifies it in polynomial time.

Algorithm:

```
Algorithm NDA_search(A, n)
{
    j := choice(1...n);
    if A[j] = x then
    {
        write(j);
        success();
    }
    write(0);
    failure();
}
```

B. Non-Deterministic Sorting Algorithm

In theory, a non-deterministic sorting algorithm "guesses" a sorted permutation and checks if it's correct.

Algorithm:

```
Algorithm NDA_sort(A, n)
{
    for i := 1 to n do
        B[i] := 0;

    for i := 1 to n do
    {
        j := choice(1...n);
        if B[j] ≠ 0 then
            failure();
        B[j] := A[i];
    }

    for i := 1 to n - 1 do
        if B[i] > B[i + 1] then
            failure();

    write(B[1...n]);
    success();
```

```
}
```

---

17. Define Hamiltonian Cycle. Write the algorithm for the same. (4 M)

Definition:

A **Hamiltonian cycle** in a graph is a cycle that **visits every vertex exactly once** and returns to the starting vertex.

Algorithm:

```
Algorithm H-cycle(k)
{
    repeat
    {
        NextValue(k);
        if (x[k] = 0) then
            return;
        if (k = n) then
            if (x[n] is adjacent to x[1]) then
                write(x[1:n]);
        else Hamilton(k + 1);
    } until (false);
}
```

---

18. Explain the following. (12 M)

1) NP-Hard and NP-Complete

- **NP-Hard** problems are those that are at least as hard as the hardest problems in NP (nondeterministic polynomial time). These problems **may or may not be in NP**, meaning they might not be decision problems, and their solutions may not be verifiable in polynomial time. If any NP-Hard problem can be solved in polynomial time, then every problem in NP can be solved in polynomial time too. Example: Travelling Salesman Problem (TSP) in optimization form is NP-Hard.

- **NP-Complete** problems are a subset of NP-Hard problems. They are **both in NP and NP-Hard**, which means that (1) they are decision problems whose solutions can be verified in polynomial time, and (2) every other problem in NP can be polynomially reduced to them. If any NP-Complete problem is solved in polynomial time, all NP

problems can be solved in polynomial time. Example: Satisfiability (SAT) problem, 0/1 Knapsack problem.

---

2) Characteristics of NP Problems

- The primary characteristic of an NP problem is that **its solution can be verified in polynomial time** even if finding that solution is hard.
- NP problems are typically **decision problems**, i.e., their output is “yes” or “no.”
- They can be solved in polynomial time by a **non-deterministic Turing machine**, which can "guess" the right solution.
- Many real-world complex problems belong to NP, including job scheduling, route optimization, and boolean satisfiability.

---

3) Non-deterministic Sorting

- Non-deterministic sorting is a theoretical concept used to illustrate how a non-deterministic machine would sort data.
- It "guesses" a permutation of the input and then verifies two conditions: (1) whether the permutation is sorted and (2) whether it is a valid rearrangement of the input.
- If both conditions are satisfied, the algorithm accepts the permutation as a sorted result. This process demonstrates verification in polynomial time.

4) Non-deterministic Searching

- Similar to sorting, non-deterministic searching is a theoretical approach where the machine "guesses" the index of the target element in the array.
- It then checks if the element at the guessed index matches the target. If yes, it returns success; otherwise, failure.
- This model is used to explain the behavior of **NP problems**, where finding the solution may be hard, but verifying it is fast.

---

19. Write Short Notes on (12 M)

i) P Class Problems: The class of decision problem but ar solvale in O(p(n)) time where p(n) is a polunomial of problems input size n. Examples of problems in P include Binary Search, Matrix Multiplication, and Dijkstra's Algorithm.

ii) NP-Hard: Class of decision problems whoes proposed solution can be verified in polynomial time and solvable by a non - deterministic polynomial algorithm. Complexity is non - quadratic and is controlled by O(2**n.n**2).

iii) NP-Complete: Refered as NP complete if it is in NP and as hard as any problem in NP, there is no known efficient way to locates a solution to such a problem.
Time required to solve the problem using any currently known algorithm increases very quickly as the size of the problem grows. E.g. graph coloring problem.

iv) Deterministic Algorithm

- A deterministic algorithm is one which, for a given input, always produces the **same output** with the same sequence of steps.
- These algorithms follow a **predictable and repeatable path** of execution and do not involve guessing.
- Examples include Merge Sort, Bubble Sort, and Binary Search.

---

23. Differentiate between recursive and iterative algorithm design. (4 M)

Feature	Recursive Algorithm	Iterative Algorithm
Definition	Solves a problem by solving smaller subproblems recursively.	Solves a problem using looping constructs like `for` or `while`.

Memory Usage	Uses **stack memory** due to recursive function calls.	Uses **less memory**, no function call overhead.
Ease of Implementation	Easier for problems with natural recursive structure (e.g., Tree Traversals).	Often more efficient and avoids overhead.
Termination	Must have a **base condition** to prevent infinite calls.	Uses loop control conditions.

---

25. Write non-deterministic algorithm to generate CLIQUE of size k from the graph of n vertices. (5 M)

```
Algorithm Clique(G, n, k, S)
{
    // Size of clique = k
    // S represents a set in which vertices are sorted.

    S = ∅  // S is initially empty set

    for i = 1 to k do
    {
        t = choice(1...n)

        if (t ∈ S) then
            failure();
        else
            S = S ∪ {t}  // t distinct vertices are selected
    }

    // At this point S contains k distinct vertex indices
```

```
    for all pairs (i, j) such that i ∈ S, j ∈ S and i ≠ j do
    {
        if ((i, j) ∉ E) then
            failure();
    }

    success();
}
```

---

26. Explain decision and optimization problems. (4 M)

Decision Problems: Any problem for which the answer is zero or one is called as decision problem. An algorithm for a decision problem is termed as decision algorithm.

Optimization Problems: Any problem that involves the indentification of an optimal values (maximum or minimum) of a given cost function is known as optimization problem. An optimization algorithm is used to solve an optimisation problem.

28. Explain following NP-problems with respect to graph. (12 M)

i) CLIQUE Problem: A **clique** is an undirected graph G=(V,E). A clique is a subset of $V' \subseteq V$ of vertices, each pair of which is connected by an edge in E. In other words, a **clique is a complete subgraph** of G.

The **size of a clique** is the number of vertices it contains. The **clique problem** is the optimization problem of finding a clique of maximum size in a graph

Clique={(G,k):G is a graph with a clique of size k}

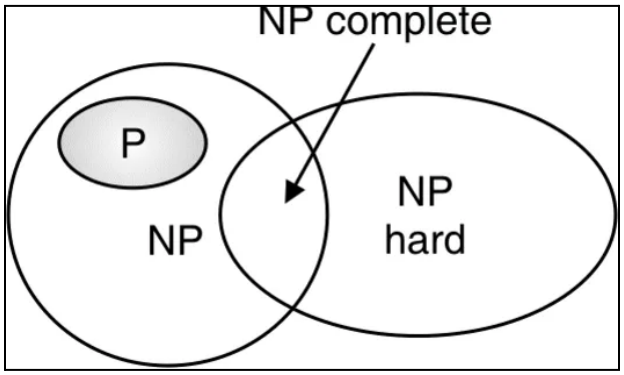
ii) Independent Set Problem: The graph is divided into to **set of vertices S1 and S2**, the vertices in S1 are **not connected** with each other. The vertices present in S1 will have **independent connectivity** with all the vertices present in S2.

iii) Graph Partitioned into Triangles: GPT represent clique of size 3 i.e. there will be 3 vertices in the clique. It is an **NP problem**, as a solution can be verified in polynomial time by checking whether all subsets indeed form triangles and cover all vertices.

---

29. Prove that P ⊆ NP (5 M)

The above statement is true with the following justification:

Out of the class of problems, for only some problems, it is possible to formulate the decision and implement them.

The class of problems for which input is known but decision implementations are variable is NP. If out of this class for some problems, it is possible to implement the decision in polynomial time, then the class is called a polynomial class.
This justifies that **P** ⊆ **NP**.